

Informações Gerais do Projeto Prático #02

Condições de conclusão

Estudantes, servidores e a comunidade externa frequentemente precisam consultar documentos institucionais da UNIPAMPA (resoluções, calendário acadêmico, projetos pedagógicos de curso, etc.) para responder dúvidas do dia a dia. Esses documentos são extensos, dispersos em diferentes portais e nem sempre fáceis de navegar.

A ideia do Projeto Prático #02 é construir o UniBot: um assistente conversacional que responde perguntas sobre a UNIPAMPA com base em documentos institucionais, usando RAG (Retrieval-Augmented Generation) para recuperar trechos relevantes do conteúdo e gerar respostas fundamentadas.

O assistente deve manter **memória da conversa** (para acompanhar o contexto ao longo do diálogo) e **memória de longo prazo** (para lembrar de preferências e informações do usuário entre sessões diferentes). Quando uma pergunta não puder ser respondida com base nos documentos, o assistente deve recorrer a um **fallback de busca externa**.

IMPORTANTE: O sistema deve ser implementado em Python com o framework Agno, conforme material disponibilizado na disciplina.

Mais detalhes sobre o sistema a ser desenvolvido podem ser encontrados no enunciado do projeto ([link aqui](#)).

Informações Gerais

1. Diferente do Projeto Prático #01, **cada grupo deve criar seu próprio repositório no GitHub**. O repositório deve: (i) **Ser privado**, tendo como colaboradores somente os(as) integrantes do grupo e, **obrigatoriamente, o professor (usuário paulosevero)**. (ii) **Apresentar um README.md** contendo, além das instruções de instalação, execução e descrição do projeto, junto de **um mapeamento entre os usernames do GitHub e os nomes dos integrantes do grupo (com matrícula)**, para facilitar a avaliação e evitar ambiguidade em casos de usernames não óbvios.
2. O trabalho deve ser realizado em **grupos de 4 a 6 estudantes**.
3. A definição dos grupos é de responsabilidade dos estudantes. **Os grupos devem ser formados até 06/06/2026 às 23h59min**. Como a criação e o gerenciamento do repositório ficam a cargo do grupo, **atrasos na formação dos grupos não resultarão em extensão do prazo de entrega**. O prazo de entrega não terá flexibilização por conta do limite de tempo do semestre.
4. Os entregáveis do Projeto Prático #02 são:
 1. **Vídeo de apresentação** do projeto (demonstração do sistema funcionando e explicação da implementação, com duração entre 10 e 15 minutos).
 2. **Release no GitHub** com código-fonte e demais artefatos do projeto.
5. **A entrega deve ser feita até o dia 14/06/2026 às 23h59min (horário de Brasília)**.

1. Entregas após esse prazo não serão aceitas (nota zero para todo o grupo).
2. O grupo deverá realizar a entrega enviando em anexo dois links:
 1. Link para o vídeo de apresentação (hospedado em plataforma de vídeo, e.g., YouTube, Vimeo, etc.)
 2. Link para a release no GitHub (repositório com código-fonte e demais artefatos do projeto).
3. O envio deve ser feito por meio do formulário disponível no Moodle a ser disponibilizado em momento oportuno.

Critérios de Avaliação

Os critérios de avaliação são divididos em dois grupos:

1. Avaliação por Grupo (totalizando 50% da nota):

1. Funcionalidade do sistema (30%): O sistema atende a todos os requisitos definidos no enunciado?
2. Qualidade do código (20%): O projeto segue as boas práticas de Engenharia de Software definidas no enunciado?

2. Avaliação Individual (totalizando 50% da nota):

1. Domínio técnico (25%): O estudante demonstra compreensão dos conceitos aplicados no projeto?
2. Contribuição individual (25%): O estudante contribuiu comprovadamente (com base em commits com conteúdo significativo distribuídos de forma equilibrada ao longo do tempo de desenvolvimento) para a implementação do projeto?

Enunciado Projeto Prático #02

Condições de conclusão

Descrição

Estudantes, servidores e a comunidade externa frequentemente precisam consultar documentos institucionais da UNIPAMPA (resoluções, calendário acadêmico, projetos pedagógicos de curso, etc.) para responder dúvidas do dia a dia. Esses documentos são extensos, dispersos em diferentes portais e nem sempre fáceis de navegar.

A ideia do Projeto Prático #02 é construir o UniBot: um assistente conversacional que responde perguntas sobre a UNIPAMPA com base em documentos institucionais, usando RAG (Retrieval-Augmented Generation) para recuperar trechos relevantes do conteúdo e gerar respostas fundamentadas.

O assistente deve manter **memória da conversa** (para acompanhar o contexto ao longo do diálogo) e **memória de longo prazo** (para lembrar de preferências e informações do usuário entre sessões diferentes). Quando uma pergunta não puder ser respondida com base nos documentos, o assistente deve recorrer a um **fallback de busca externa**.

IMPORTANTE: O sistema deve ser implementado em Python com o framework Agno, conforme material disponibilizado na disciplina.

Requisitos Funcionais Mínimos

- 1. Ingestão do corpus:** o sistema deve ter um pipeline documentado que carrega os documentos institucionais, aplica uma estratégia de chunking e indexa o conteúdo em uma base vetorial.
- 2. Recuperação e resposta (RAG):** o assistente deve responder perguntas recuperando os trechos mais relevantes do corpus e gerando uma resposta fundamentada neles. A resposta deve indicar as fontes/trechos utilizados (por exemplo, nome do documento e seção).
- 3. Fallback de busca externa:** o assistente deve detectar quando a base de documentos não cobre a pergunta e, nesses casos, recorrer a uma busca externa, deixando claro na resposta que a informação veio de fonte externa, e não do corpus institucional.
- 4. Memória de conversa:** o assistente deve manter o histórico da conversa dentro de uma sessão, permitindo perguntas de acompanhamento que dependem do contexto anterior.
- 5. Memória de longo prazo:** o assistente deve persistir informações relevantes do usuário (por exemplo, curso, campus, preferências) e recuperá-las em sessões futuras.
- 6. Interface:** o sistema deve oferecer uma interface funcional e apresentável para a interação com o usuário.

Observações sobre Tecnologias

- **Fallback:** o uso do Tavily é uma sugestão, não uma obrigação. O grupo pode optar por outra API de busca, por outra ferramenta de recuperação externa ou por outra abordagem que julgar mais adequada, desde que cumpra o requisito de complementar as respostas quando o corpus não for suficiente.

- **Interface:** o Streamlit é uma sugestão. Grupos com mais familiaridade com outras bibliotecas ou frameworks (por exemplo, Gradio, React, Svelte, etc.) podem utilizá-los, desde que o resultado final seja uma aplicação funcional e apresentável.

Práticas Obrigatórias de Engenharia de Software

- **Organização**

- Repositório com estrutura de arquivos e diretórios clara e organizada.
- Arquivo README.md com descrição do projeto, instruções de instalação/execução, descrição da estrutura de diretórios e mapeamento entre usernames e nomes+matrículas. No README.md, também deve estar presente uma seção descrevendo as decisões de RAG: estratégia de chunking adotada, modelo de embeddings utilizado, parâmetros de recuperação e o critério usado para acionar o fallback.
- .gitignore adequado.
- Gerenciamento de dependências baseado em pyproject.toml.
- .env para variáveis de ambiente.
- Documentos do corpus versionados no repositório ou, quando isso não for viável (por exemplo, por tamanho), instruções claras de como obtê-los e ingeri-los.
- Comando ou script de ingestão documentado, de forma que o avaliador consiga reconstruir a base vetorial a partir do zero.

- **Clean Code**

- Código homogêneo em estilo, idioma e formatação (PEP8 para Python, em inglês)
- Funções curtas (preferivelmente com no máximo 20 linhas)
- Nomes descritivos (variáveis, funções, classes)
- Type hints e docstrings em todas as assinaturas

- **Princípios de Desenvolvimento**

- Single Responsibility Principle (SRP): Cada classe/função com uma única responsabilidade
- Open/Closed Principle (OCP): Aberto para extensão, fechado para modificação; evite blocos condicionais extensos e use polimorfismo
- Interface Segregation Principle (ISP): Prefira interfaces/classes abstratas específicas
- Dependency Inversion Principle (DIP): Dependenda de abstrações, não de implementações concretas; use injeção de dependências